

Review of linked list and resizing array

Linked list

A (*singly*) *linked list* is a data structure that consists of a sequence of elements, where each element points to the next element in the sequence. The first element is called the *head* of the list, and the last element points to `nullptr`. In a *doubly* linked list, each element points to both the next and the previous elements. It provided the following operations:

Operation	Description
<code>push_front(x)</code>	Insert an item x at the head of the list
<code>push_back(x)</code>	Insert an item x at the tail of the list
<code>pop_front()</code>	Remove the head of the list
<code>size()</code>	Return the number of items in the list
<code>empty()</code>	Return true if the list is empty

All operations take constant time.

Resizing array

A *resizing array* is an array that automatically resizes itself when the number of elements exceeds its capacity. When the array is full, a new array of double the size is created, and all elements are copied to the new array. This operation takes n steps, where n is the number of elements in the array. Resizing arrays are useful when the number of elements is not known in advance. Similarly, when the number of elements in the array is less than a quarter of its capacity, a new array of half the size is created, and all elements are copied to the new array.

It provided the following operations:

Operation	Description
<code>push_back(x)</code>	Insert an item x at the end of the array
<code>pop_back()</code>	Remove the last item of the array
<code>size()</code>	Return the number of items in the array
<code>empty()</code>	Return true if the array is empty

All operations take constant time in the worst case, except `push_back(x)` and `pop_back()` which take linear time in the worst case. However, the amortized time for `push_back(x)` and `pop_back()` is constant. By amortized, we mean that if we perform a sequence of m operations, the total time taken by all operations is $O(m)$, which means that the average time taken by each operation is constant.

Stack and Queue

Stack and *queue* are two important ADTs. A stack stores items in a last-in, first-out (LIFO) manner, whereas a queue stores items in a first-in, first-out (FIFO) manner. Following table summarizes the operations of a

stack and a queue.

Stack	Queue	Description
<code>push(x)</code>	<code>enqueue(x)</code>	Insert an item x
<code>pop()</code>	<code>dequeue()</code>	Remove and return an item (most recently inserted item for stack and least recently inserted one for queue)
<code>empty()</code>	<code>empty()</code>	Return true if the stack/queue is empty
<code>size()</code>	<code>size()</code>	Return the number of items in the stack/queue

We have seen two implementations of stacks and queues in the previous semester: using linked list and using resizable array. Note that from the perspective of the user, the two implementations of stack (respectively queue) are indistinguishable as both provide the identical public interface. The performance of the two implementations is different, however. The following table summarizes the time complexity of the operations of stack and queue using singly linked list and resizable array.

	Implementation	Time Complexity
Stack	Singly linked list	$O(1)$ for all operations
	Resizing array	$O(1)$ for all operations (amortized)
Queue	Singly linked list	$O(1)$ for all operations
	Resizing array	$O(1)$ for all operations (amortized)

Lab 1: Text Editor Buffer with Stacks (Array vs Linked List)

Learning Objectives

- Review linked list and resizing array data structures.
- Implement stack ADT on top of both data structures.
- Use stacks to build a text editor buffer.
- Compare time/space trade-offs between implementations.

Part A: Resizing Array

Implement a class **ResizingArray** that implements a dynamic array of **chars**. The class should use a resizing array to store the elements. Use the code provided in **resizable-array.hpp** file as a starting point. You need to complete the implementation of the methods. You must handle edge cases, such as popping from an empty array or accessing an index out of bounds.

Part B: Linked List

Implement a class **LinkedList** that implements a singly linked list of **chars**. The class should use a linked list to store the elements. Use the code provided in **linked-list.hpp** file as a starting point. You need to complete the implementation of the methods. You must handle edge cases, such as deleting from an empty list.

Part C: Stack ADT

Implement stack ADT twice using **ResizingArray** and **LinkedList** as the underlying data structure. The classes should provide the operations described in the previous section and use the respective data structure to store the elements. Use the code provided in **stack.hpp** file as a starting point. You need to complete the implementation of the methods. You must handle edge cases, such as popping from an empty stack.

Part D: Text Editor Buffer

Let say we want to implement a text editor buffer that allows the user to insert and delete characters at the current cursor position. The cursor can be moved left or right, and the text can be retrieved as a string. If we use a string to store the text, then inserting or deleting a character at the current cursor position will take linear time in the worst case. We need to use a data structure that allows us to insert and delete characters at the current cursor position in constant time.

There are several data structures that can be used to implement a text editor buffer, such as linked list, gap buffer, and rope. One simple idea is to use two stacks to store the characters to the left and right of the cursor. The left stack will store the characters to the left of the cursor, and the right stack will store the characters to the right of the cursor. When we insert a character at the current cursor position, we push it onto the left stack. When we delete a character at the current cursor position, we pop it from the left stack. When we move the cursor left, we pop a character from the left stack and push it onto the right stack. When we move the cursor right, we pop a character from the right stack and push it onto the left stack. When we retrieve the text, we concatenate the characters in both stacks.

You will now simulate a text editor buffer with two stacks (left of cursor, right of cursor). Implement `TextBuffer` twice: Using stacks based on `ResizableArray` and using stacks based on `textttLinkedList`.

The class should provide the following operations:

Operation	Description
<code>insert(c)</code>	Insert a character <i>c</i> at the current cursor position
<code>remove()</code>	Delete the character at the current cursor position
<code>moveLeft()</code>	Move the cursor one position to the left
<code>moveRight()</code>	Move the cursor one position to the right
<code>getText()</code>	Return the current text in the buffer as a string

The class should use two stacks to store the characters to the left and right of the cursor. Use the code provided in `text-buffer.hpp` file as a starting point. You need to complete the implementation of the methods.

Part E: Testing and Comparison

The provided `main.cpp` file contains a test harness that validates your text buffer implementations and compares their performance. It creates one instance of `TextBufferArray` and one of `TextBufferList`, then performs a large number of random operations—including insert, move left, move right, and remove—using an identical random sequence (with a fixed seed) on both buffers, guaranteeing a fair comparison. The `Stopwatch` class measures the total execution time for each implementation. After the operations complete, the program retrieves the final text from both buffers and compares them (they should be identical if your code is correct), then prints a timing summary and a short snippet of the resulting text. This lets you quickly verify functional correctness and observe the performance trade-offs between the array-based and linked-list-based stacks in a realistic editing scenario.

Compare the performance of the two implementations and write a short report `report.txt` summarizing your findings.

Submission

Submit the following files:

- `resizeable-array.hpp`: Complete implementation of resizable array class.
- `linked-list.hpp`: Complete implementation of linked list class.
- `stack.hpp`: Complete implementation of stack class using resizable array and linked list.
- `text-buffer.hpp`: Complete implementation of text buffer class using stacks based on resizable array and linked list.
- `report.txt`: A short report comparing the two approaches.